Sobre o projeto de automação - API e Interface de usuário

O que é este projeto

Esse projeto reúne testes automatizados para duas aplicações de demonstração. O Restful Booker é uma API usada para testar operações de reservas de hotel. O Sauce Demo é um site de compras fictício, onde testei o login, o carrinho e a finalização de um pedido.

A ideia é repetir algumas verificações automaticamente sempre que o código for executado. Assim, fica mais fácil perceber quando um comportamento que já funcionava deixa de funcionar depois de alguma mudança.

O que eu testei

Na API, testei a autenticação e o ciclo de vida de uma reserva: criar, consultar, atualizar e excluir. Também incluí tentativas que deveriam ser recusadas, como consultar uma reserva inexistente ou alterar e excluir uma reserva sem um token válido.

No Sauce Demo, testei o login, a inclusão de um produto no carrinho, a conferência do item e o checkout até a confirmação da compra. Também verifiquei login com dados incorretos, o bloqueio do usuário locked_out_user, a validação do primeiro nome no checkout e o logout.

Exemplos dos fluxos

No teste da API, primeiro crio uma reserva e guardo o ID que a própria API retorna. Uso esse ID para consultar a reserva, alterar alguns dados e confirmar que a alteração foi salva. No fim, excluo o registro e verifico que ele não existe mais. Isso evita depender de uma reserva fixa que pode mudar ou ter sido criada por outra pessoa.

No site, o teste entra com um usuário válido, adiciona uma mochila ao carrinho, preenche os dados de checkout, confere o resumo e finaliza a compra. Outro teste deixa o primeiro nome em branco e confere se o site mostra a mensagem de validação esperada.

Como organizei o código

Separei os testes de API dos testes de interface. Para o site, usei Page Objects, que guardam os seletores e as ações de cada tela. Por exemplo, a classe de login sabe quais campos preencher e qual botão clicar; o teste fica responsável por conferir se o resultado foi o esperado.

Para a API, criei os clientes AuthApi e BookingApi, que centralizam as chamadas HTTP. Não usei Page Objects nessa parte porque não estamos interagindo com páginas visuais. Também usei TypeScript para deixar mais claros os formatos dos dados enviados e recebidos.

Relatório e execução automática

O Mochawesome gera um relatório HTML com o resultado dos testes e, quando disponível, gráficos e screenshots. Os relatórios ficam na pasta cypress/reports e não são enviados junto com o código, porque são arquivos criados a cada execução.

Também configurei uma esteira no GitHub Actions. Ela roda quando há um push na branch main, um pull request direcionado à main ou quando é iniciada manualmente. A esteira prepara o Node.js, instala as dependências, executa os testes e tenta guardar o relatório e as screenshots como artefatos por 14 dias. O resultado de cada execução pode ser visto na aba Actions do repositório.

Como entender uma falha

Quando um teste passa, quer dizer que o comportamento observado correspondeu ao que o teste esperava naquele cenário. Isso ajuda, mas não garante que todo o sistema esteja sem problemas: a automação só cobre os casos que foram escritos.

Quando um teste falha, pode ser um defeito, uma mudança no comportamento que exige atualizar o teste, uma instabilidade do serviço externo ou um problema no ambiente. O relatório ajuda a localizar o que aconteceu, mas é preciso analisar a falha antes de concluir que existe um problema no produto.

Limitações e próximos passos

O Restful Booker é público e compartilhado, então outras pessoas podem alterar os dados e o serviço pode ficar instável. O Sauce Demo também depende de um serviço externo. Essas situações podem afetar uma execução sem que tenha havido mudança no nosso código.

O usuário locked_out_user foi usado para confirmar que o acesso é bloqueado; como ele não consegue entrar, não é possível usar esse perfil para testar uma compra completa. No checkout, por enquanto, testei especificamente a ausência do primeiro nome.

Com mais tempo, eu ampliaria as validações dos demais campos do checkout, testaria outros dados inválidos na API e exploraria mais perfis de usuário. A prioridade dependeria do risco e do que fosse mais importante para o negócio.